

Лабораторная работа #5-6 – ES6, ES8, ES2017. Проектирование типов. Классы. Наследование

Цель работы: изучить основы проектирования собственных типов/классов.

Теоретическая часть

Что такое ECMAScript?

Сначала немного истории. JavaScript создавался как скриптовый язык для Netscape. После чего он был отправлен в ECMA International для стандартизации (ECMA – это ассоциация, деятельность которой посвящена стандартизации информационных и коммуникационных технологий). Это привело к появлению нового языкового стандарта, известного как ECMAScript.

Последующие версии JavaScript уже были основаны на стандарте ECMAScript. Проще говоря, ECMAScript – стандарт, а JavaScript – самая популярная реализация этого стандарта.

Что такое ES? История версий.

ES – это просто сокращение для ECMAScript. Каждое издание ECMAScript получает аббревиатуру ES с последующим его номером. Всего существует 8 версий ECMAScript. ES1 была выпущена в июне 1997 года, ES2 – в июне 1998 года, ES3 – в декабре 1999 года, а версия ES4 – так и не была принята. Не будем углубляться в эти версии, так как они морально устарели, а рассмотрим только последние четыре.

ES5 был выпущен в декабре 2009 года, спустя 10 лет после выхода третьего издания. Среди изменений можно отметить:

- поддержку строгого режима (strict mode);
- аксессоры getters и setters;
- возможность использовать зарезервированные слова в качестве ключей свойств и ставить запятые в конце массива;
- многострочные строковые литералы;
- новую функциональность в стандартной библиотеке;
- поддержку JSON.

Версия ES6/ES2015 вышла в июне 2015 года. Это также принесло некую путаницу в связи с названием пакета, ведь ES6 и ES2015 – это одно и то же. С выходом этого пакета обновлений комитет принял решение перейти к ежегодным обновлениям. Поэтому издание было переименовано в ES2015, чтобы отражать год релиза. Последующие версии также называются в соответствии с годом их выпуска. В этом обновлении были сделаны следующие изменения:

- добавлено [деструктурирующее присваивание](#);
- добавлены [стрелочные функции](#);
- в [шаблонных строках](#) можно объявлять строки с помощью ` (обратных

кавычек). Шаблонные строки могут быть многострочными, также могут интерполироваться;

- [let](#) и [const](#) – альтернативы var для объявления переменных. Добавлена «временная мертвая зона»;
- [итератор и протокол итерации](#) теперь определяют способ перебора любого объекта, а не только массивов. [Symbol](#) используется для присвоения итератора к любому объекту;
- добавлены [функции-генераторы](#). Они используют yield для создания последовательности элементов. Функции-генераторы могут использовать yield* для делегирования в другую функцию генератора, кроме этого, они могут возвращать объект генератора, который реализует оба протокола;
- добавлены [промисы \(Promise\)](#).

ES2016 (ES7) вышла в июне 2016 года. Среди изменений в этой версии ECMAScript можно отметить:

- оператор возведения в степень **;
- метод Array.prototype.includes, который проверяет, содержится ли переданный аргумент в массиве.

Спустя еще год выходит версия ES2017 (ES8). Данный стандарт получил следующие изменения:

- асинхронность теперь официально поддерживается ([async/await](#));
- «висячие» запятые в параметрах функций. Добавлена возможность ставить запятые в конце списка аргументов функций;
- добавлено два новых метода для работы со строками: padStart() и padEnd(). Метод padStart() подставляет дополнительные символы слева, перед началом строки. А padEnd(), в свою очередь, справа, после конца строки;
- добавлена функция Object.getOwnPropertyDescriptors(), которая возвращает массив с дескрипторами всех собственных свойств объекта;
- добавлено разделение памяти и объект Atomics.

Что же касается ES.Next, то этот термин является динамическим и автоматически ссылается на новую версию ECMAScript. Стоит отметить, что каждая новая версия приносит новые функции для языка.

Проектирование типов. Классы. Наследование

Практически с момента возникновения языка JavaScript его синтаксис и шаблоны разработки навевали мысли о возможной поддержке классов. Благодаря таким вещам, как операторы new и instanceof, а также свойству .constructor, возникает соблазн считать, что в JS существуют классы, скрытые где-то внутри системы прототипов.

Разумеется, «классы» JS не имеют ничего общего с обычными классами.

Ключевое слово class. Хотя механизм прототипов в JS не умеет работать как традиционные классы, это не уменьшает количество просьб расширить синтаксическое

удобство до такой степени, чтобы представление о «классах» больше совпадало с реальностью. Рассмотрим появившееся в ES6 ключевое слово **class** и связанный с ним механизм.

Эта функциональная особенность появилась после бурных и продолжительных дебатов и стала компромиссом между несколькими противоположными точками зрения на подход к реализации классов в JS. Большинство разработчиков, мечтавших о полноценных классах, сочтут новый синтаксис весьма привлекательным, хотя и обнаружат отсутствие важных составляющих. Но поводов для волнения тут нет. Комитет TC39 уже работает над дополнительными функциональными особенностями, которые улучшат классы в следующих версиях стандарта.

В основе механизма классов, появившегося в ES6, лежит ключевое слово **class**, идентифицирующее блок, содержимое которого определяет члены прототипа функции.

Рассмотрим пример:

```
class Foo {
  constructor(a,b) {
    this.x = a;
    this.y = b;
  }
  gimmeXY() {
    return this.x * this.y;
  }
}
```

Здесь нужно обратить внимание на следующие вещи.

- Запись **class Foo** влечет за собой создание (специальной) функции с именем **Foo**, во многом напоминающей те, что создавались разработчиками до появления ES6.
- **constructor(..)** определяет сигнатуру функции **Foo(..)** и содержимое ее тела.
- Для методов класса применяется тот же самый синтаксис «кратких методов», что и для объектных литералов. Сюда же входит краткая форма генератора, а также синтаксис методов чтения/записи из ES5. Но методы класса не являются перечисляемыми, в то время как методы объекта перечисляемы по умолчанию.
- В отличие от объектных литералов, в теле класса отсутствуют запятые, отделяющие члены друг от друга. Более того, запятые там вообще недопустимы.

Определение синтаксиса для ключевого слова **class** в предыдущем фрагменте можно грубо представить как существовавший до ES6 эквивалент:

```
function Foo(a,b) {  
    this.x = a;  
    this.y = b;  
}  
Foo.prototype.gimmeXY = function() {  
    return this.x * this.y;  
}
```

Как в предшествующей ES6 форме, так и в новой «класс» допускает создание экземпляров, которые используются точно так, как следовало бы ожидать:

```
const f = new Foo( 5, 15 );  
f.x; // -> 5  
f.y; // -> 15  
f.gimmeXY(); // -> 75
```

Ключевые слова extends и super. Классы ES6 имеют удобный синтаксис для установления делегирующей связи [[Prototype]] между двумя прототипами функций, часто неправильно называемой «наследованием» или (по непонятной причине) «наследованием прототипов». Для этого используется ориентированная на классы терминология, – ключевое слово **extends**:

```
class Bar extends Foo {  
    constructor(a,b,c) {  
        super( a, b );  
        this.z = c;  
    }  
    gimmeXYZ() { return super.gimmeXY() * this.z; }  
}  
const b = new Bar( 5, 15, 25 );  
b.x; // -> 5  
b.y; // -> 15  
b.gimmeXYZ(); // -> 1875
```

А вот слово **super** – совершенно новое, оно позволяет делать вещи, которые до появления ES6 было невозможно реализовать напрямую (без кое-каких не слишком хороших и сложно достижимых компромиссов). В конструкторе это ключевое слово автоматически ссылается на «родительский конструктор». В предыдущем примере он назывался **Foo(..)**. Внутри метода **super** ссылается на «родительский объект», обеспечивая доступ к свойству/методу, например **super.gimmeXY()**.

Конструктор подкласса. Для классов и подклассов конструкторы не требуются; если конструктор по умолчанию опустить, в обоих случаях он будет замещен. Однако вид замещенного конструктора по умолчанию зависит от того, с каким классом – непосредственным или расширенным – мы имеем дело.

В частности, используемый по умолчанию конструктор подкласса автоматически вызывает родительский конструктор и передает любые аргументы туда. Другими словами, этот конструктор можно представить примерно вот так:

```
constructor(...args) {
  super(...args);
}
```

Отметим важную деталь. Не во всех языках программирования, поддерживающих классы, конструктор подклассов автоматически вызывает родительский конструктор. В C++ подобное имеет место, а вот в Java – нет. Еще важнее тот факт, что в версиях классов, существовавших до ES6, подобного тоже не было.

ES6 на конструкторы подклассов накладывает еще одно странное ограничение. Доступ к ключевому слову **this** в таком конструкторе появляется только после вызова метода **super(..)**. Это поведение имеет крайне сложные причины, но все главным образом упирается в тот факт, что именно родительский конструктор создает и инициализирует значение **this** для вашего экземпляра. До ES6 это работало в обратную сторону: объект **this** создавался «конструктором подклассов», а затем вызывался родительский конструктор в контексте **this** этого подкласса.

Ключевое слово static. Когда подкласс **Bar** расширяет родительский класс **Foo**, объект **Bar.prototype** через **[[Prototype]]** связывается с объектом **Foo.prototype**. Кроме того, дополнительно функция **Bar()** связывается через **[[Prototype]]** с функцией **Foo()**.

Тем не менее это бывает весьма полезно в случае, когда мы объявляем статические методы (не только свойства) для класса, так как они добавляются непосредственно в его объект-функцию, а не в объект-прототип этой функции. Рассмотрим пример:


```

class Foo {
    static cool() { console.log( "cool" ); }
    wow() { console.log( "wow" ); }
}
class Bar extends Foo {
    static awesome() {
        super.cool();
        console.log( "awesome" );
    }
    neat() {
        super.wow();
        console.log( "neat" );
    }
}
Foo.cool(); // -> "cool"
Bar.cool(); // -> "cool"
Bar.awesome(); // -> "cool" -> "awesome"

const b = new Bar();

b.neat(); // -> "wow" -> "neat"
b.awesome; // -> undefined
b.cool; // -> undefined

```

Следует помнить, что статические члены находятся в цепочке прототипа класса. По сути, они формируют параллельную цепочку между конструкторами функций.

Приватные и защищённые методы и свойства. Один из важнейших принципов объектно-ориентированного программирования – разделение внутреннего и внешнего интерфейсов.

Защищённое свойство «waterAmount». Создадим простой класс для описания кофеварки:

```

class CoffeeMachine {
    waterAmount = 0; // количество воды внутри

    constructor(power) {
        this.power = power;
        alert( `Создана кофеварка, мощность: ${power}` );
    }
}

// создаём кофеварку
const coffeeMachine = new CoffeeMachine(100);

// добавляем воды
coffeeMachine.waterAmount = 200;

```

Прямо сейчас свойства **waterAmount** и **power** публичные. Изменим свойство **waterAmount** на защищённое, чтобы иметь больше контроля над ним. Например, запретим, чтобы кто-либо устанавливал его ниже нуля.

Защищённые свойства обычно начинаются с префикса «**_**». Защищённые свойства обычно начинаются с префикса «**_**».

```

class CoffeeMachine {
    _waterAmount = 0;

    set waterAmount(value) {
        if (value < 0) throw new Error("Отрицательное количество воды");
        this._waterAmount = value;
    }

    get waterAmount() {
        return this._waterAmount;
    }
}

```

```

    constructor(power) {
        this._power = power;
    }
}

// создаём новую кофеварку
const coffeeMachine = new CoffeeMachine(100);

// устанавливаем количество воды
coffeeMachine.waterAmount = -10; // Error: Отрицательное количество воды

```

Приватное свойство «#waterLimit». Есть новшество в языке JavaScript, которое почти добавлено в стандарт: оно добавляет поддержку приватных свойств и методов.

Приватные свойства и методы должны начинаться с «#». Они доступны только внутри класса.

Например, в классе ниже есть приватное свойство **#waterLimit** и приватный метод **#checkWater** для проверки количества воды:

```

class CoffeeMachine {
    #waterLimit = 200;

    #checkWater(value) {
        if (value < 0) throw new Error("Отрицательный уровень воды");
        if (value > this.#waterLimit)
            throw new Error("Слишком много воды");
    }
}

let coffeeMachine = new CoffeeMachine();

// снаружи нет доступа к приватным методам класса
coffeeMachine.#checkWater(); // Error
coffeeMachine.#waterLimit = 1000; // Error

```


На уровне языка «#» является специальным символом, который означает, что поле приватное. Нельзя получить к нему доступ извне или из наследуемых классов.

Приватные поля не конфликтуют с публичными. Может быть два поля одновременно – приватное **#waterAmount** и публичное **waterAmount**.

МИТ СО

Дополнительный материал

1. [Классы \(javascript.ru\)](#)
2. [Промисы, async/await \(javascript.ru\)](#)
3. [Генераторы, продвинутая итерация \(javascript.ru\)](#)

Практическая часть

1. **+2 балла.** Успешно завершить тест: <https://www.freecodecamp.org/learn/javascript-algorithms-and-data-structures/#es6>
2. **+4 балл.** Выполнить задание согласно вашему варианту (см. документ).
3. **+4 балл.** Выполнить задание согласно вашему варианту (см. документ).

МИИТСО